A PROJECT REPORT ON

AI-BASED CODE DOCUMENTATION GENERATION WORKSPACE TOOL

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Abhishek Sachin Bhosale	Exam No: B400050341
Kedar Sudam Pawar	Exam No: B400050545
Abhishek Vidyasagar Ulagadde	Exam No: B400050620
Aditya Govinda Mahajan	Exam No: B400050484

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY

2024-2025

CERTIFICATE

This is to certify that the project report entitled

AI-BASED CODE DOCUMENTATION GENERATION WORKSPACE TOOL

Submitted by

Abhishek Sachin Bhosale	Exam No: B400050341
Kedar Sudam Pawar	Exam No: B400050545
Abhishek Vidyasagar Ulagadde	Exam No: B400050620
Aditya Govinda Mahajan	Exam No: B400050484

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. M. V. Mane** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. M. V. Mane
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure to present this report on our B.E. project, titled ***"AI Based Code Documentation Generation Workspace Tool"*** *First and foremost, we would like to take this opportunity to express our sincere gratitude to our project guide,* ***Prof. M. V. Mane****, for her invaluable support, guidance, and encouragement throughout the course of this project. Her insights and constructive feedback have been instrumental in shaping the direction of our work, and we are truly grateful for her time, patience, and commitment in helping us navigate the complexities of the subject.*

We would also like to extend our thanks to the Department of Computer Engineering at ***Pune Institute of Computer Technology*** *for providing us with the opportunity and resources to work on this project. The knowledge and skills imparted throughout our B.E. course has been crucial in the successful completion of this project, and we are thankful to the faculty for their constant support*

We are also grateful to ***Dr G. V. Kale****, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

Abhishek Bhosale
Aditya Mahajan
Abhishek Ulagadde
Kedar Pawar
(B.E. Computer Engg.)

ABSTRACT

In modern software development, maintaining accurate and up-to-date documentation is a critical challenge, particularly for large-scale projects with complex codebases spanning multiple programming languages and modules. Traditional methods of generating documentation often fail to capture the intricate relationships between code files and their dependencies, resulting in fragmented or incomplete information. Additionally, large functions and classes may exceed the processing limits of current large language models (LLMs), further complicating documentation generation. The proposed solution addresses these challenges by implementing an intelligent code chunking approach, which ensures that large code blocks are split into manageable sections without losing context or meaning. This method is combined with module import resolution and dependency tracking, allowing the system to generate documentation in a structured and sequential manner, ensuring that the context from external and internal module imports is utilized effectively. By employing topological sorting of dependencies, the system ensures that files are processed in the correct order, with imported modules being documented first, followed by dependent files. The solution leverages the capabilities of LLMs while overcoming their token limits, ensuring comprehensive, accurate, and context-aware documentation generation for large projects. This approach is scalable, language-agnostic, and capable of handling projects of varying complexity. The report details the methodology, implementation strategies, and real-world applications of this solution, providing a clear pathway to improving code documentation workflows in complex development environments.

Keywords: *Intelligent Code Chunking, Automated Documentation Generation, Large Language Models (LLMs), Topological Sorting, Context-aware Documentation, Multi-language Support, Code Dependency Management, Sequential Documentation Workflow.*

Contents

List of Figures

CHAPTER 1

INTRODUCTION

1.1 Overview

The AI-Based Code Documentation Generation Workspace Tool is designed to automate the process of generating comprehensive, accurate, and context-aware documentation for large and multi-language codebases. It addresses common challenges faced by developers, such as understanding unfamiliar code, maintaining up-to-date documentation, and managing complex inter-module dependencies. The system leverages intelligent code chunking, module dependency resolution, and integration with large language models (LLMs) to produce high-quality documentation. With both a command-line interface and a web-based platform, the tool supports seamless interaction, version control integration, and scalability, making it ideal for enterprise, open-source, and agile development environments.

1.2 Motivation

In the ever-evolving world of software development, maintaining high-quality documentation is crucial for effective collaboration, scalability, and long-term maintenance of codebases. As projects grow in complexity, with developers working across various frameworks, programming languages, and systems, the need for accurate, up-to-date, and comprehensive documentation becomes even more essential.

However, the process of manually generating documentation is often time-consuming, prone to human error, and unable to keep up with frequent code changes. Additionally, as development teams increasingly rely on code reuse through internal and external module imports, the relationships between different files and dependencies become more complex to manage and document. Large codebases, including those with intricate interdependencies, require an efficient system for automatically generating documentation that can handle both the size and complexity of the project.

Motivated by the need to streamline documentation generation and ensure accuracy across large and complex projects, this report explores the development of a solution that combines intelligent code chunking with module dependency resolution to create a scalable, automated, and context-aware documentation system.

1.3 Problem Definition and Objectives

1.3.1 Problem Definition

To develop a platform that automates code documentation process, minimizing manual effort and keeping information consistent and up-to-date. The platform will integrate with the codebase to generate and update documentation automatically, reducing reliance on scattered, outdated tools. It will streamline onboarding, improve productivity, and enhance team collaboration by making code easier to understand and manage.

1.3.2 Objectives

1. Automated Documentation Generation: The primary objective is to automate the process of generating comprehensive and context-aware documentation for large codebases, reducing the manual effort required to write and maintain documentation.

2. Handling Large Codebases: Develop an intelligent code chunking algorithm that can break down large code files into manageable sections, ensuring that even code exceeding LLM token limits is properly documented without losing context.

3. Dependency Resolution: Implement a module dependency resolver that can accurately identify and process both internal and external module imports, generating documentation in the correct order and preserving the context from imported modules.

4. Multi-language Support: Ensure that the system supports multiple programming languages and adapts the documentation generation process to handle language-specific syntax and structures, making the tool flexible and applicable across diverse projects.

1.4 Project Scope & Limitations

1.4.1 Project Scope

This project aims to overcome key limitations identified in current literature, such as limited semantic understanding, poor context handling across files, and inconsistent multi-language support. By leveraging intelligent code chunking, dependency resolution, and LLM integration, the tool ensures accurate, context-rich documentation generation. It addresses the gaps in existing solutions by focusing on scalability, language-agnostic processing, and real-time documentation updates for complex and evolving codebases.

1.4.2 Limitations

Traditional documentation generation methods face several limitations, particularly in the context of large codebases. These challenges include:

1. **Handling Large Code Blocks :** Modern large language models (LLMs), such as GPT-4, have a limited context window, typically allowing for only 8,000 to 32,000 tokens per session. When dealing with large functions or classes that exceed these limits, the documentation generated can be incomplete or fragmented.

2. **Maintaining Context Across Files :** Many projects rely on both internal and external module imports, and the relationship between these imported classes or functions and the main code can be complex. If the documentation is generated in an incorrect order, with the main code being processed before its dependencies, the generated documentation lacks the necessary context.

3. **Complex Dependency Structures :**Large projects often consist of multiple interconnected modules and libraries. When generating documentation, it is crucial to ensure that the documentation for imported modules is generated first, so that it can be referenced and used for the main code. Failure to do so results in incomplete and inaccurate documentation.

1.5 Methodologies of Problem solving

To address the above mentioned problems, a comprehensive approach is needed that:

1. Chunks large code files intelligently to fit within the LLM's context window, ensuring each chunk is meaningful and well-documented.
2. Handles module imports and dependencies by ensuring the correct order of documentation generation, where dependencies are processed first, and context is preserved across the documentation process.
3. Generates accurate and comprehensive documentation for codebases of varying sizes and complexity, across multiple programming languages.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

2.1.1 Automatic documentation generation

The authors of the paper[1] introduced a technique for automatically generating descriptive comments for Java methods by analyzing method signatures, control flow, and naming conventions. Their work was an important step toward bridging the gap between code semantics and documentation but it was only limited to the specific languages and predefined templates.

2.1.2 Evaluating large language models trained on code

The authors of the paper[3] introduced Codex, an LLM fine tuned on large-scale code repositories, which demonstrated the ability to generate high-quality documentation and even assist in code completion tasks. Codex was specifically trained to understand both the syntax and semantics of various programming languages, making it a powerful tool for generating contextually relevant documentation across diverse languages.

2.1.3 CodeBERT: A pre-trained model for programming and natural languages

The authors of the paper[4] developed CodeBERT, a model trained on paired code and natural language data to improve code understanding and documentation tasks. The authors showed that CodeBERT could accurately generate documentation for both small and large code snippets, though it still faced challenges when dealing with large functions or complex module dependencies.

2.1.4 Machine learning for big code and naturalness

The authors of the paper[5] proposed a method for automatically summarizing large code functions by splitting them into smaller, more digestible pieces and then using neural models to generate summaries. Their approach to hierarchical summarization highlighted the need for maintaining context across multiple chunks to ensure a coherent overall documentation.

2.1.5 Hierarchical Code Graph Summarization (HCGS)

The paper[6] introduces Hierarchical Code Graph Summarization (HCGS), a novel method that builds structured, multi-layered summaries of large codebases using a bottom-up traversal of code graphs. By leveraging language-agnostic analysis and parallel processing, HCGS significantly improves code retrieval accuracy, outperforming traditional code-only approaches—especially in large, complex repositories

2.1.6 Mining version histories to guide software changes

The authors of the paper[8] proposed dependency tracking mechanisms that automatically map inter-module relationships within a project. It applies data mining to version histories in order to guide programmers along related changes.

2.2 Existing methodologies

2.2.1 Manual Documentation

Developers write documentation manually alongside code. Often error-prone, inconsistent, and time-consuming. Example: Using inline comments or Markdown files like README.md maintained manually.

2.2.2 Docstring-Based Tools

Tools like Sphinx (for Python) or Javadoc (for Java) generate documentation from in-code comments. Requires developers to write structured docstrings consistently.

2.2.3 Static Analysis Tools

Analyze code without execution to infer documentation. Tools like Doxygen or Pydoc extract structure but lack deep contextual understanding.

However, these above mentioned methods often struggle with scalability, cross-language support, and real-time adaptability, highlighting the need for more flexible and intelligent solutions.

2.3 Research Gap Analysis

After going through refernce papers and material we identified following drawbacks in existing solutions:

1. Limited Semantic Context Understanding: Many models miss deeper meaning in code, leading to documentation that lack important context or details. Improved semantic processing could make these documentation more accurate.

2. Contextual Limitations of Transformer Models: Transformer-based models work well for code summarization but often struggle with context beyond the code snippet itself, such as code dependencies, resulting in incomplete or overly generalized summaries.

3. Evaluation Metrics and Benchmarks: There is a lack of standardized evaluation metrics that accurately reflect code summary quality. While benchmarks like CodexGLUE provide a starting point, there is no universal standard for measuring the readability, conciseness, and accuracy of generated summaries

4. Handling Multi-Language CodeBases and Change Tracking: Summarizing multi-language codebases is inconsistent, and models rarely consider historical changes in code when generating docs.

2.4 Scope of Improvement

Based on the reviewed literature and research gap analysis, several areas remain open for improvement in current automated code documentation solutions. Firstly, most models exhibit limited semantic understanding, often failing to capture deeper code intent, leading to generic or incomplete summaries.

Additionally, transformer-based models, while effective on isolated snippets, lack the contextual awareness required to document interdependent modules accurately. Finally, existing tools struggle with multi-language support and change tracking, resulting in inconsistent documentation across diverse codebases and versions.

Addressing these challenges can significantly enhance the accuracy, reliability, and applicability of future documentation tools.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

3.1.1 Assumptions

1. The codebase being processed has a well-defined structure with no circular dependencies.
2. The user has provided access to external and internal modules, and the system can access these files.
3. Developers have integrated the tool correctly with their project to track changes and regenerate documentation as necessary.

3.1.2 Dependencies

1. The system depends on the availability of a powerful LLM (such as GPT-4 or Codex) to process and generate documentation.
2. The system requires access to external libraries or module files when resolving imports.
3. The tool depends on regular expression-based or AST-based parsing libraries to identify code structures in different programming languages (Python, C++, Java, JavaScript).

3.2 Functional Requirements

3.2.1 System Feature 1: Intelligent Code Chunking

- Description: The system shall break down large code files into smaller, manageable chunks to fit within the LLM's context window, ensuring that meaningful documentation is generated without exceeding token limits.
- Inputs: Large code files in various programming languages.
- Process: Identify logical boundaries such as functions, classes, and loops. Split the code while maintaining contextual integrity. Generate summaries for each chunk and combine them into a higher-level summary for the full file.
- Outputs: Detailed documentation for each chunk and a high-level summary for the entire file.
- Acceptance Criteria: The documentation is generated without any incomplete or missing portions, even for large code files.

3.2.2 System Feature 2: Module Dependency Resolution

- Description: The system shall identify and resolve module dependencies, ensuring that documentation is generated in the correct order where imported modules are documented first, followed by the files that depend on them.
- Inputs: Code files with internal and external module imports.
- Process: Analyze the code files to identify import statements. Generate documentation for the imported modules. Use the context from the module documentation when generating documentation for dependent files.
- Outputs: Documentation that accurately captures the relationships between modules and their usage in the main code.
- Acceptance Criteria: The system correctly documents both imported modules and dependent code files without missing any dependencies.

3.2.3 System Feature 3: Multi-Language Support

- Description: The system shall support documentation generation for multiple programming languages, including Python, C++, Java, and JavaScript.
- Inputs: Code files in various languages.
- Process: Detects the programming language for each file. Apply language-specific parsing rules to identify functions, classes, and imports. Generate documentation specific to the syntax and structure of each language.
- Outputs: Documentation for all files in the correct language, following language-specific conventions.
- Acceptance Criteria: The tool generates correct and readable documentation across multiple programming languages without errors.

3.2.4 System Feature 4: Documentation Context Passing

- Description: The system shall pass context from previously generated documentation of modules to the main code files when generating documentation, ensuring continuity.
- Inputs: Documentation summaries from imported modules.
- Process: Use the generated documentation for imported modules as input/context for dependent files. Incorporate relevant details from imports into the documentation for the main code.
- Outputs: Comprehensive documentation that references and explains imported modules in the context of the main code.
- Acceptance Criteria: The documentation generated for dependent files incorporates relevant details from the imported modules, providing a cohesive explanation.

3.3 External Interface Requirements

3.3.1 User Interfaces

1. Command Line Interface : Users will interact with the system primarily via a CLI tool, where they can initiate the documentation generation process for specific files or the entire codebase. The CLI will allow for options to specify language preferences, directories, and generation modes (full vs. incremental).

2. Web Interface : A web platform, a user-friendly dashboard may allow users to view, edit, and manage generated documentation. Users can navigate through files and access contextualized documentation for different modules.

3.3.2 Software Interfaces

1. LLM API Integration : The system will interface with external LLM services such as OpenAI's GPT-4 or Codex through their APIs. The software must handle token limits and API response errors efficiently.

2. Version Control Integration : Integration with version control systems like Git may allow for documentation to be generated based on specific branches or commits.

3. Parsing Libraries : The tool will interface with libraries like ast for Python, and equivalent parsing tools for C++, Java, and JavaScript to analyze and chunk code.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. Documentation Generation Time: The system should be capable of generating documentation for large codebases within a reasonable time frame, ensure low latency when processing code chunks.
2. Scalability: The system must handle large-scale projects efficiently, supporting both small codebases and enterprise-level projects with complex dependencies.

3.4.2 Safety Requirements

1. Data Integrity: The system should only read files and never modify or delete code ensuring that all code files and modules remain unchanged.
2. Failure Handling: If the system encounters an error during documentation generation it should provide meaningful error messages and logs.

3.4.3 Security Requirements

1. API Security: All API interactions must be secured.
2. Access Control: Only authorized users should be able to generate or view documentation for certain projects.
3. Data Privacy: Code data processed by the system must remain secure.

3.4.4 Software Quality Attributes

1. Functionality: The tool must deliver documentation for given codebase.
2. Reliability: The system should consistently generate correct documentation across different runs.
3. Usability: WebUI/CLI must allow devs to easily trigger, view, and manage documentation.
4. Maintainability: Architecture should easily support new language, improvements to chunking and parsing algorithms.

3.5 System Requirements

3.5.1 Database Requirements

1. Document Storage: The system should have a database to store generated documentation. The storage can be simple file-based storage.

2. Dependency Tracking: A database will be required to store information on the dependencies between code files and modules, ensuring that the system can track and process imports and module usage accurately.

3.5.2 Software Requirements (Platform Choice)

1. Software Stack:

    - Backend: Python (for parsing, chunking, and processing code files), integrated with an LLM API such as OpenAI's GPT-4 or Together AI.
    - Database: MongoDB for scalable storage, or PostgreSQL for relational data storage, depending on project size.
    - Web Platform: If integrated with a web-based platform, the web backend uses Django (Python), and the frontend useing React.
    - Git: For tracking code changes.

2. External Services:

    - LLM API Integration: OpenAI GPT-4, Together AI, Groq apis for generating context-aware documentation.
    - Version Control Integration: Github apis for tracking code changes and generating documentation for specific commits or branches.

3.5.3 Hardware Requirements

1. The system is designed to run on standard development machines for local use.

2. For scalability and deployment in production environments, we may leverage AWS cloud infrastructure.

3.6 Analysis Models: SDLC Model to be applied

For this project, the Agile Software Development Life Cycle (SDLC) model will be applied. Agile is chosen for its flexibility, iterative development approach, and ability to accommodate changes in requirements and scope over time. The following phases will be applied within the Agile framework:

1. **Requirements Gathering (Sprint 1):**
   In this phase, user stories will be collected and prioritized. Key features like intelligent code chunking, module resolution, and multi-language support will be broken down into actionable tasks for development.

2. **Design and Prototyping (Sprint 2-3):**
   A working prototype will be designed based on the prioritized requirements, focusing on the intelligent code chunking algorithm and initial module import handling.

3. **Development (Sprint 4-9):**
   Core development will follow iterative cycles, with each sprint focusing on delivering functional increments of the system. Development tasks will include language-specific parsing, LLM integration, and multi-language support.

4. **Testing (Sprint 10-11):**
   The system will undergo various testing. Any defects identified will be fixed in subsequent sprints.

5. **Deployment and Maintenance (Sprint 12-14):**
   The system will be deployed to target environments. The project will enter the maintenance phase, ensuring bug fixes, security patches, and feature updates as necessary.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system architecture for the intelligent documentation generation tool is designed to handle large codebases with complex dependencies across multiple languages, while efficiently generating context-aware documentation. The architecture is divided into several key components:

1. CLI Tool
2. Backend Service
3. Web Workspace/Platform
4. Database
5. Version Control Integration

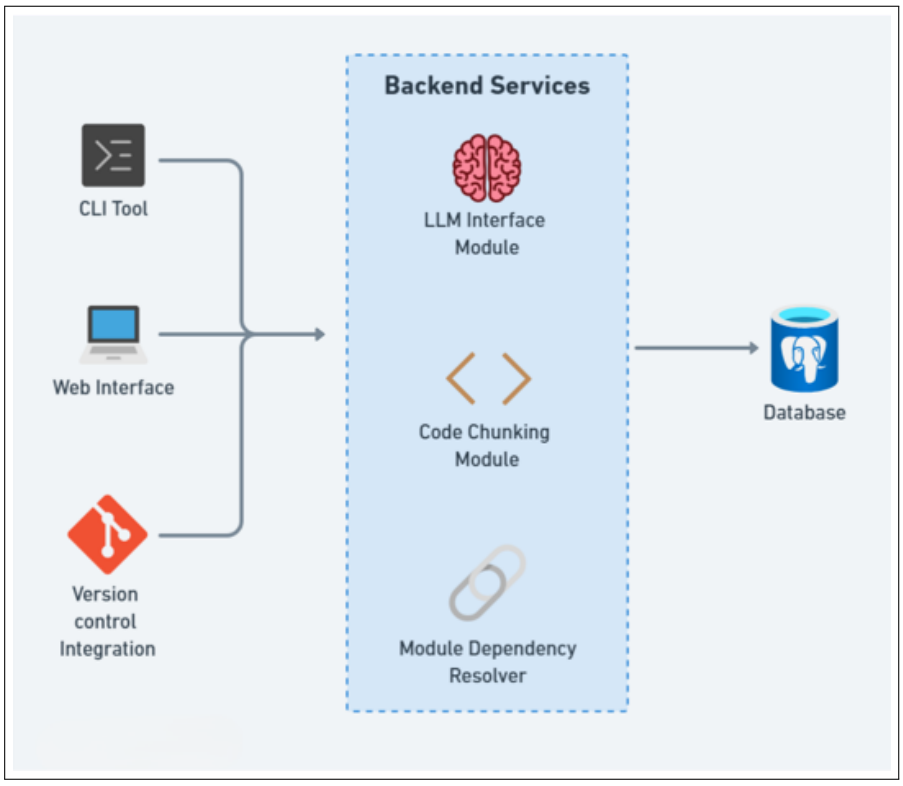


Figure 4.1: System Architecture Diagram

4.2 Data Flow Diagram

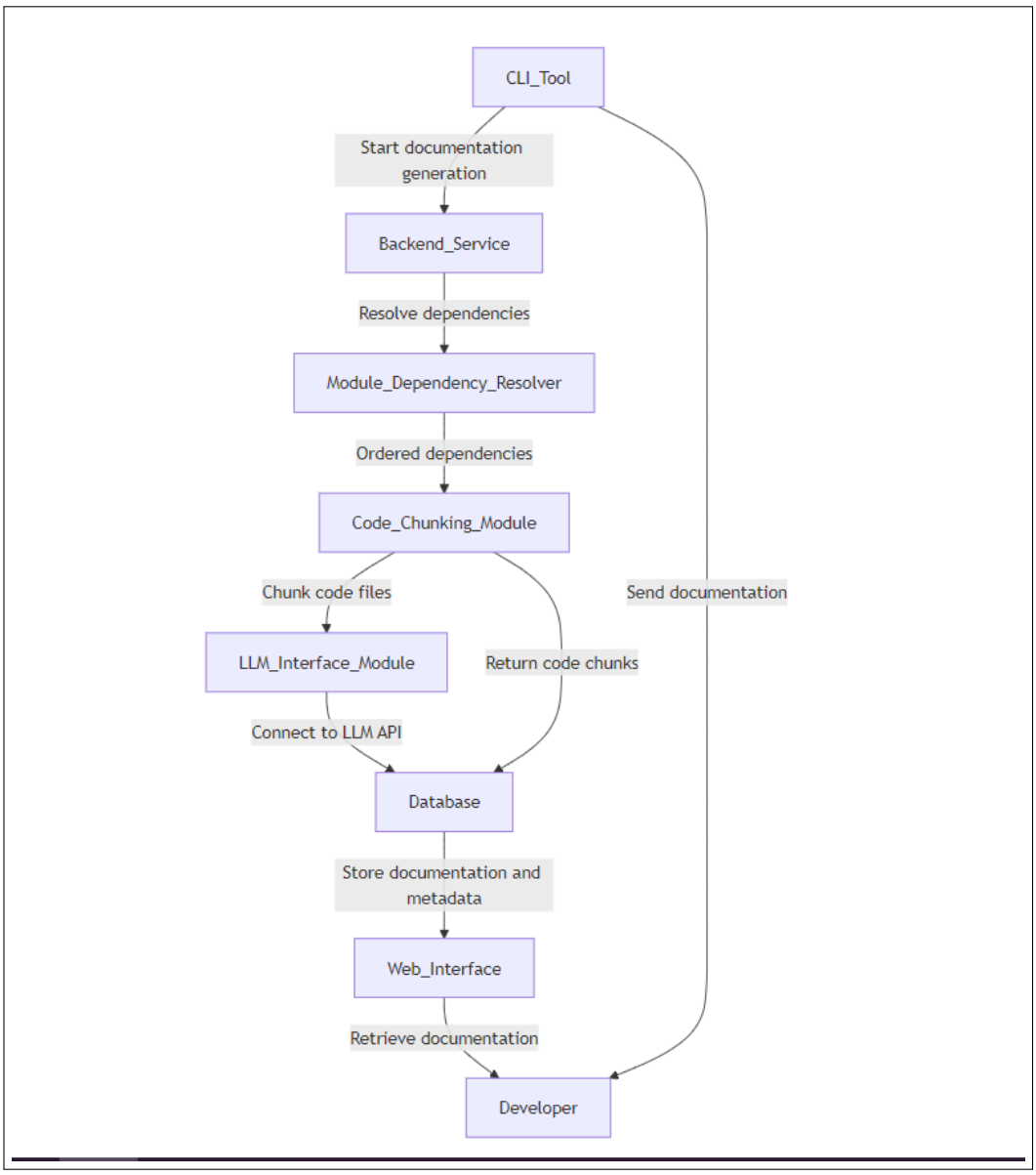


Figure 4.2: Data Flow Diagram

4.3 Entity Relationship Diagram

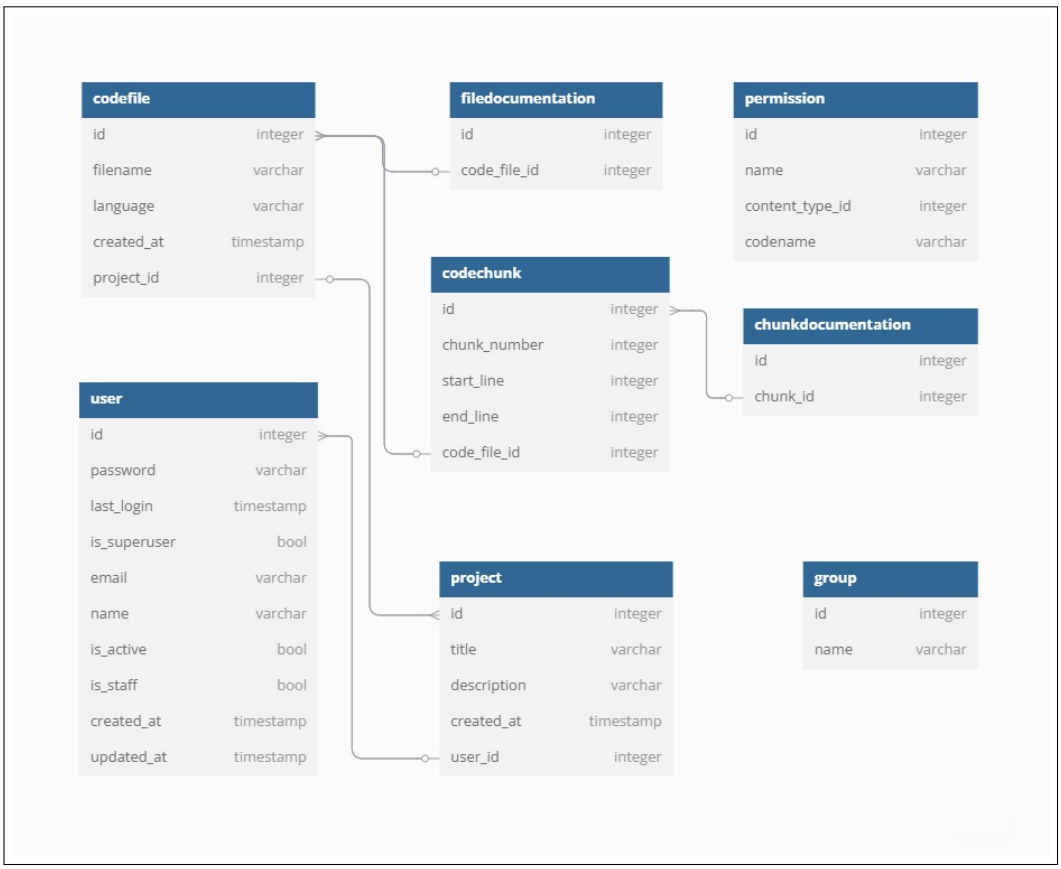


Figure 4.3: ER Diagram

4.4 UML Diagrams

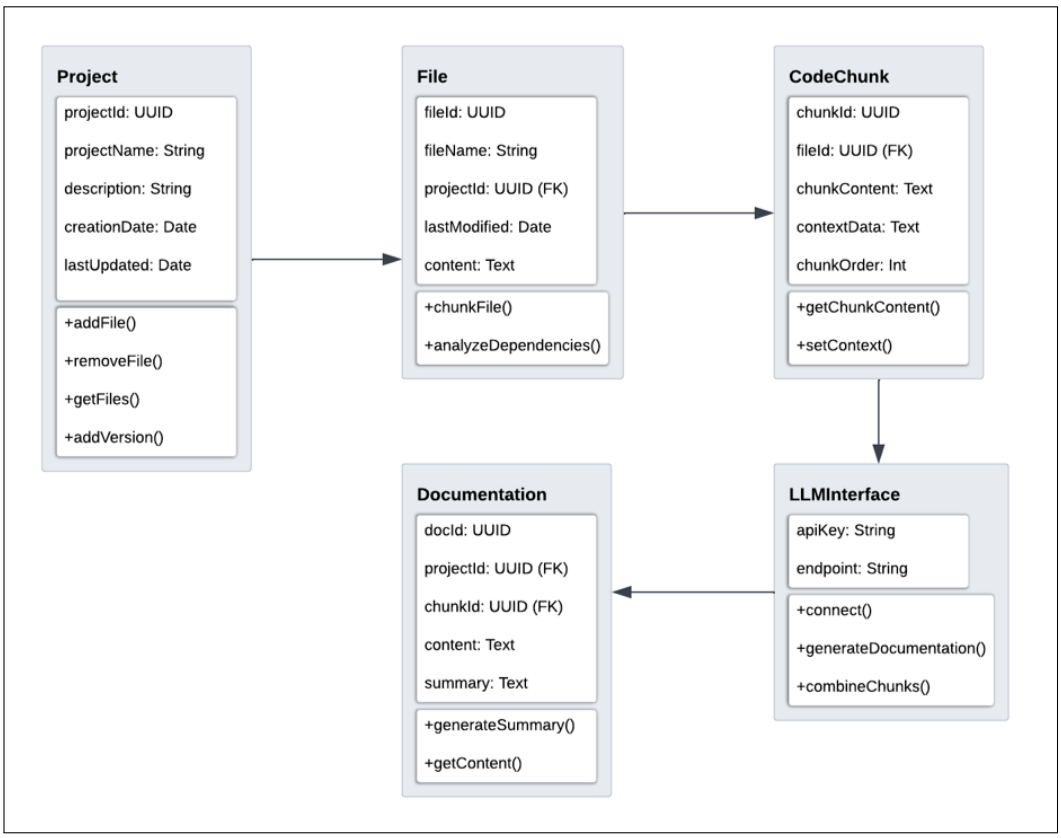


Figure 4.4: UML Diagram

4.5 Sequence Diagram

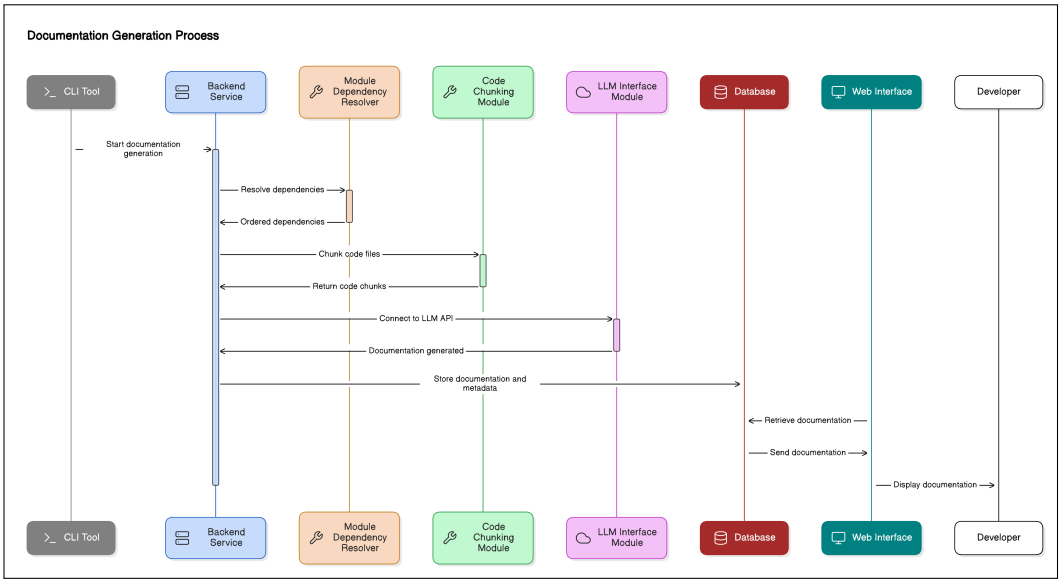


Figure 4.5: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Estimates

The development of the intelligent documentation generation tool is structured into phased efforts with estimated time and resource allocation. Phase 1, dedicated to requirements gathering and analysis, is expected to take 1 week with 2 team members focusing on scope understanding, feasibility analysis, and requirement collection. Phase 2, involving initial design and prototyping, will span 2 weeks and include 2 developers and 1 UI/UX contributor to design the system architecture and prototype key modules like chunking and module handling. The core development in Phase 3 is planned for 4 weeks, with 3 developers implementing code chunking, dependency resolution, and multi-language support (Python, C++, Java, JavaScript). In Phase 4, testing and validation will be carried out over 2 weeks by 2 developers and 2 testers, ensuring the documentation is accurate, scalable, and performs well. Finally, Phase 5, focused on deployment and maintenance, is scheduled for 1 week with 2 developers.

5.1.2 Reconciled Estimates

Considering potential delays, team availability, and additional resource needs, the revised estimates include buffer time to manage unforeseen challenges. Phase 1 is extended to 1.5 weeks to allow for thorough requirements gathering and analysis. Phase 2 is adjusted to 3 weeks, accommodating design iterations. Phase 3, with expected integration challenges across multiple languages and modules, is now estimated at 5 weeks. Due to the complexity of multi-language testing, Phase 4 is extended to 3 weeks. Lastly, Phase 5 is extended to 1.5 weeks to ensure smooth deployment and monitoring. With these adjustments, the total project timeline is estimated at 14 weeks, including buffer time.

5.2 Risk Management

5.2.1 Risk Identification

1. Language-Specific Issues: Challenges in parsing different programming languages (e.g., C++, JavaScript) may arise due to syntax variations.
2. LLM Token Limits: The project relies on LLMs with context window limitations, which could lead to incomplete documentation for large code blocks.
3. Integration Challenges: Difficulty in ensuring that the generated documentation correctly captures the interdependencies between different modules, especially with complex project structures.

5.2.2 Risk Analysis

1. Language-Specific Issues:
   - Probability: Medium
   - Impact: High (could delay multi-language support)
   - Mitigation: Use a combination of regex-based parsing and AST for language-specific handling.
2. LLM Token Limits:
   - Probability: High
   - Impact: Medium (incomplete documentation for large functions)
   - Mitigation: Intelligent code chunking to split code into manageable pieces while maintaining context across chunks.
3. Integration Challenges:
   - Probability: Medium
   - Impact: High (incorrect or incomplete documentation for dependent modules)
   - Mitigation: Use dependency resolution and topological sorting to handle module imports before processing dependent files.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- Risk Mitigation: Risks will be mitigated by incorporating robust design decisions (like language-specific parsing and chunking strategies) and ensuring test coverage for multi-language support. A fallback mechanism will be added for cases where certain dependencies or code blocks cannot be resolved automatically.
- Risk Monitoring: Weekly sprint reviews will help monitor progress, identify emerging risks, and adjust priorities as necessary. Detailed logs of performance during testing will help track any scalability or LLM token limit issues.
- Risk Management: A clear communication channel between developers, testers, and project managers will ensure quick decision-making when new risks or bottlenecks are encountered. The project manager will maintain a risk log, which will be reviewed during the development phases.

5.3 Project Schedule

5.3.1 Project Task Set

1. Requirements Gathering and Feasibility Analysis.
2. System Design, Architectural design, code chunking algorithm design, dependency resolution plan.
3. Core Development of chunking algorithm, import handling and dependency tracking and multi-lang support.
4. Testing.
5. Deployment.

5.3.2 Task Network

Requirement Analysis → Design & Prototype → Core Dev → Testing & Validation → Deployment → Maintenance

5.3.3 Timeline Chart

The project timeline is outlined below, with the estimated duration of each task:

Task	Duration (in weeks)
Requirements Gathering & Analysis	3
Design & Prototyping	3
Core Development	4
Testing and Validation	2
Deployment	1

Table 5.1: Timeline Chart

5.4 Team Organization

The success of this project relies on effective team collaboration and communication. The roles within the team are clearly defined to ensure that each member contributes effectively to the project.

5.4.1 Team structure

The team is composed of four members, each responsible for specific areas of the project:

- **Abhishek Bhosale:** Responsible for core development.
- **Aditya Mahajan:** Leads frontend and integration of services.
- **Kedar Pawar:** Focuses on system integration and testing.
- **Abhishek Ulagadde:** Handles testing, validation, documentation and deployment.

5.4.2 Management reporting and communication

Weekly meetings were held with the project guide, Prof. M. V. Mane, to report progress and discuss any challenges.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

6.1.1 Dependency Resolution Module

- Identifies internal/external dependencies within the codebase to ensure logical flow.
- Constructs a dependency graph to map file relationships and resolve module dependencies.
- Uses topological sorting on the dependency graph to establish the processing order, ensuring modules are documented before dependent files.
- Helps maintain clarity and context by managing dependencies in a structured sequence.

6.1.2 Code Chunking Module

- Divides large code files into small chunks while preserving the context of each part.
- Constructs an Abstract Syntax Tree to identify logical units within the code.
- Breaks code down into coherent sections, ensuring each chunk is comprehensive and contextually accurate.
- Prepares code chunks for efficient processing and documentation by the LLM Interface Module.

6.1.3 LLM Interface Module

- Connects to an external Large Language Model (LLM) API (e.g.,GPT-4, TogetherAI, Groq) for generating code documentation.
- Sends each code chunk to the LLM to create detailed, context-aware summaries.
- Aggregates the documentation generated for individual chunks to produce a cohesive overall summary.

6.2 Tools and Technologies Used

- **Python:** For backend processing and integration logic
- **Javascript React:** For building a responsive web interface
- **Django Rest Framework:** For API and server-side operations
- **PostgreSQL:** For storing metadata and documentation markdown files
- **LLM Integration:** External APIs like OpenAI GPT-4, Together AI, Langchain for documentation generation
- **LangChain:** Used to manage prompt engineering, context handling, and interaction workflows with large language models
- **Git:** Version control integration to track code changes and generate documentation accordingly
- **Docker:** Containerization for consistent environment setup and deployment
- **AWS Cloud Stack:** Deployment environment to handle processing, storage, and application hosting.

6.3 Algorithm Details

1. The developer triggers the documentation process using CLI/WebUI, specifying the target codebase or files to be documented.
2. The backend service receives the request, initiates the documentation pipeline across various modules.
3. The module dependency resolver scans the codebase for import statements and builds a dependency graph.
4. Uses topological sorting on the generated dependency graph to determine the correct processing order.
5. The code chunking module parses each file into an Abstract Syntax Tree (AST) and logically splits large functions or classes into smaller, context-preserving chunks.
6. Each chunk is sent to the LLM with relevant prompts and context to generate concise, human-readable documentation.
7. The generated documentation chunks are collected and merged to form a structured, coherent summary for the entire code file.
8. The backend service stores the final documentation along with related metadata (e.g., file path, chunk index) in the database for retrieval and display.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

To ensure the reliability, accuracy, and usability of the automated documentation generation tool the following types of testing were conducted:

7.1.1 Unit Testing

Unit testing focused on individual components such as the code chunking module, dependency resolver, and LLM interface will be tested in isolation to verify their correctness and robustness.

7.1.2 Integration Testing

Integration testing validates the interaction between modules—ensuring that components like chunking, dependency resolution, and documentation generation work seamlessly together.

7.1.3 Performance Testing

To evaluate the scalability of the tool, performance testing will test the system with large codebases to evaluate its response time, throughput, and scalability under varying loads.

7.1.4 Usability Testing

Usability Testing focuses on evaluating how easily and efficiently users can interact with the system. It involves observing developers using the CLI and web interface to generate, view, and manage documentation.

7.2 Test cases Test Results

Category	Description	Input	Expected Output
Unit Testing	Verify function-level code chunking	Python file with 3 functions	3 distinct function chunks identified
Unit Testing	Test module dependency resolution	main.py importing 2 local modules	Accurate dependency graph construction
Integration Testing	Validate LLM generates documentation from chunked input	Single code chunk with doc prompt	Context-aware, relevant documentation
Integration Testing	End-to-end test for multi-file project	Project with interdependent files	Ordered documentation respecting dependencies
Usability Testing	Verify that generated docs are well-structured and readable	Sample project	Markdown output is formatted, structured, and easy to navigate

Table 7.1: Test cases Test Results 1

CHAPTER 8

RESULTS

8.1 Outcomes

1. The AI-Based code documentation generation tool is successfully implemented for python, java, c++ projects.
2. Tool can handle codebases by breaking the code into smaller syntactically aware chunks.
3. Dependencies between multiple files are handled by creating topological sorting of such files.
4. Better documentation generation because of context aware system.
5. Tool provides documentation for every python/java/c++ file in project.
6. User can view code related to the file and chunks.
7. The tool provides the actual list of chunks for each file in projects. The chunk start line number and end line number are mentioned for deeper visibility.
8. Documentation is generated in a hierarchical and dependency-aware manner.
9. Improves the onboarding experience for new developers.

8.2 Screen Shots

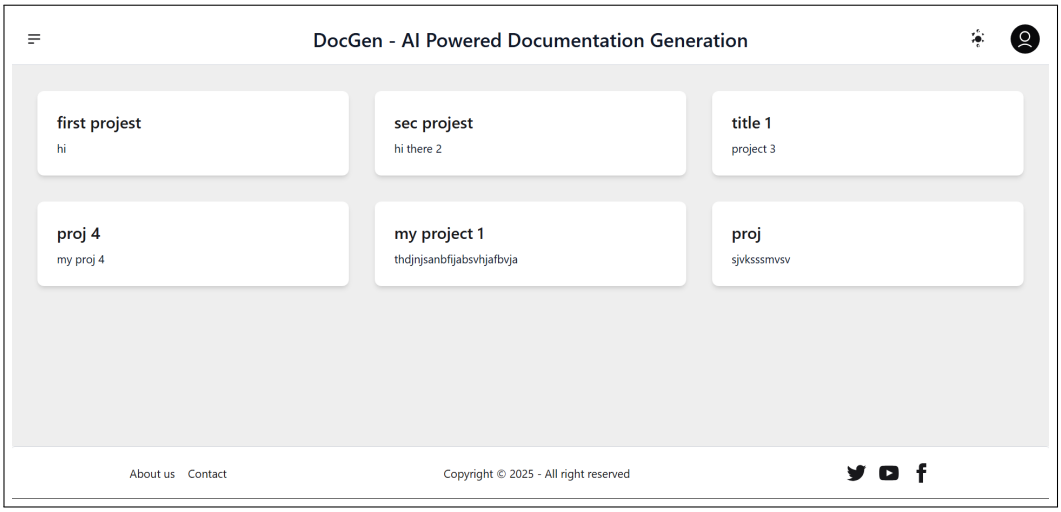


Figure 8.1: Projects

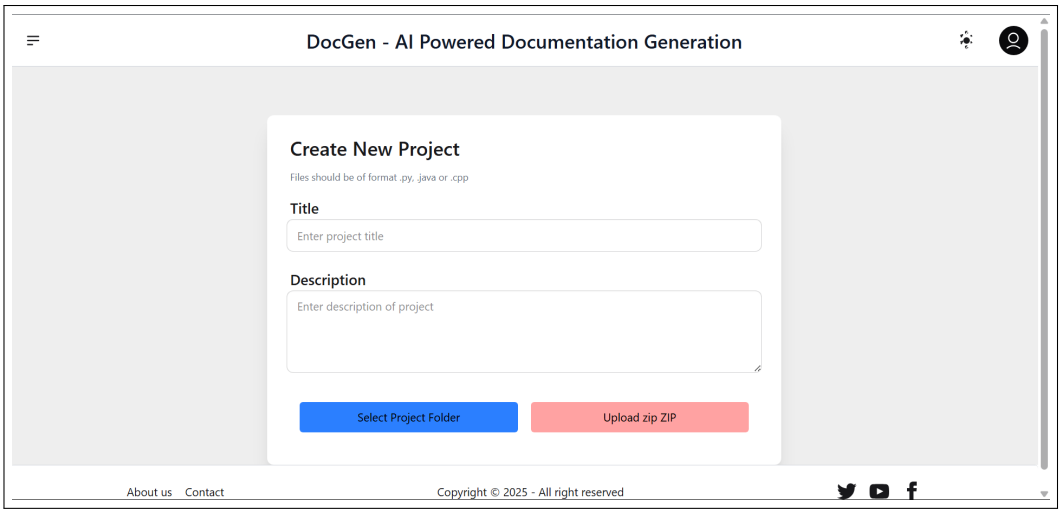


Figure 8.2: Upload new project

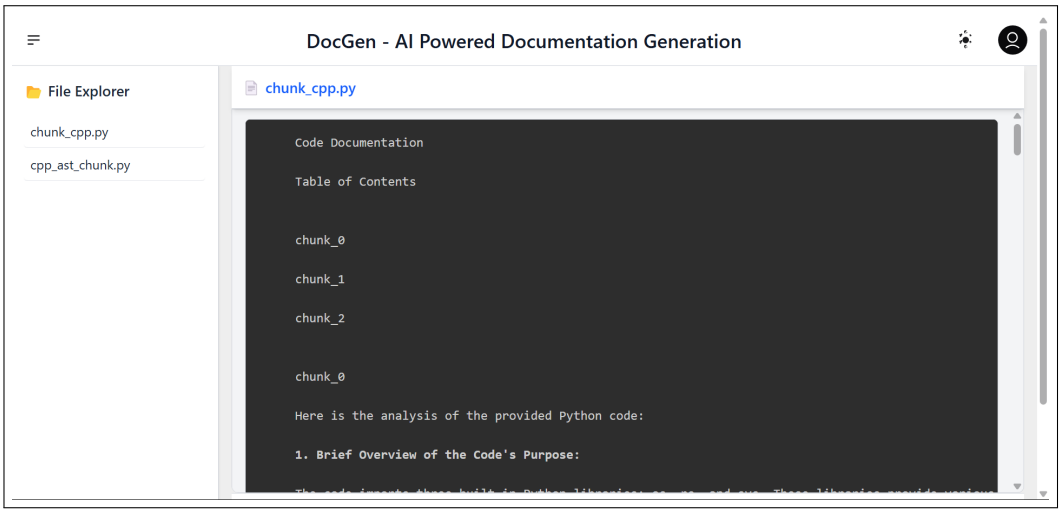


Figure 8.3: Complete documentation corresponding to each file in project

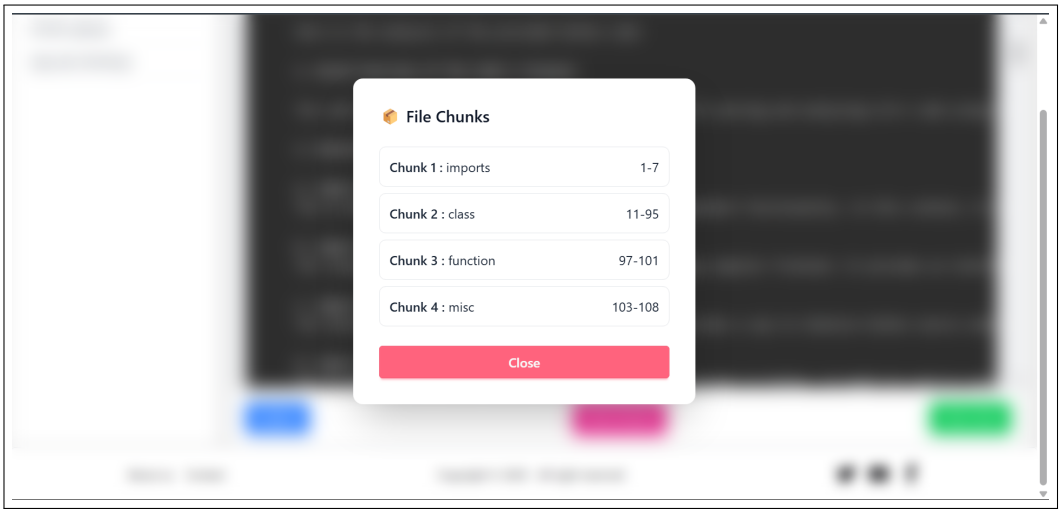


Figure 8.4: List of chunks in a file

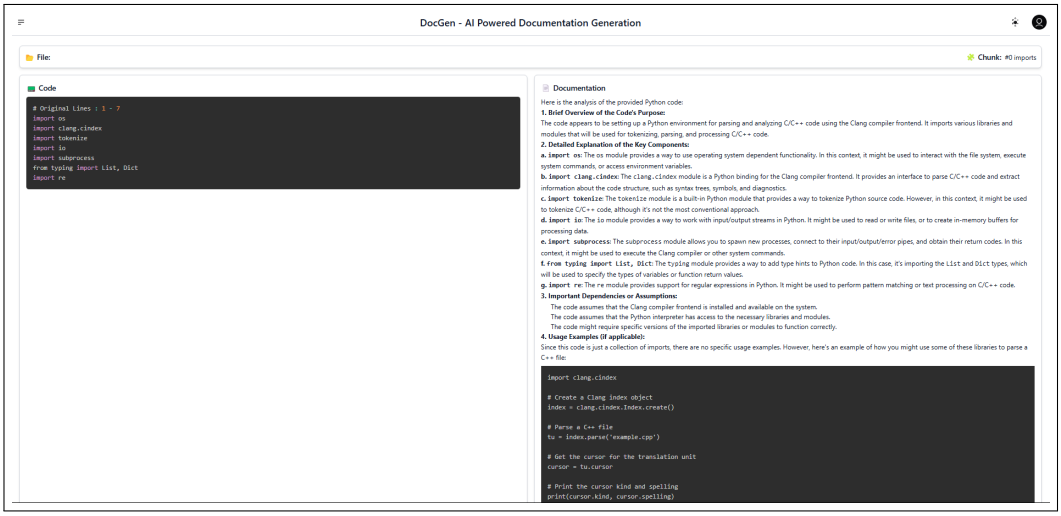


Figure 8.5: Documentation for a specific chunk

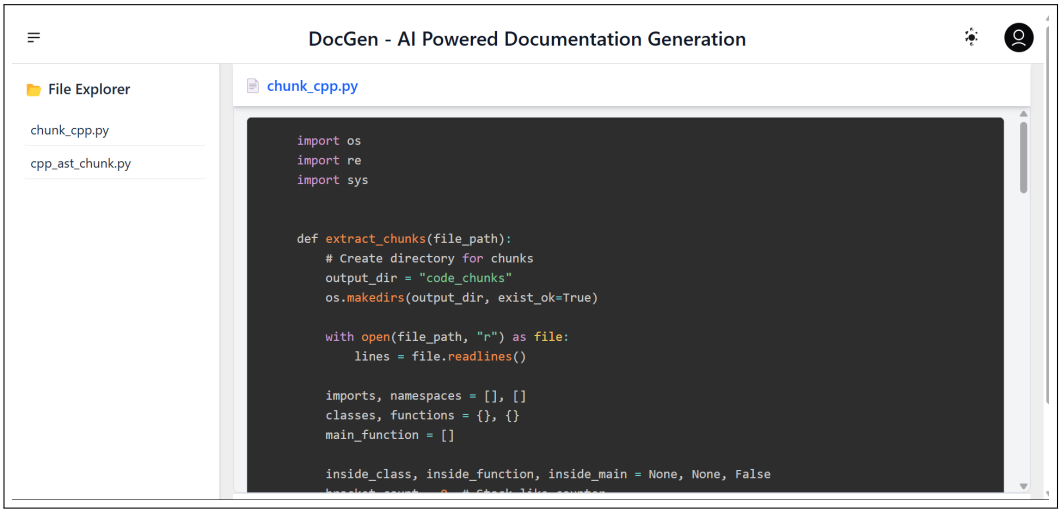


Figure 8.6: Code related to file

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

The AI-based Code Documentation Generation Workspace Tool addresses critical challenges in maintaining up-to-date, accurate documentation for large and complex codebases. By leveraging intelligent code chunking, dependency resolution, and integration with powerful language models, the system ensures that documentation is both context-aware and scalable across multiple programming languages. The tool not only automates a traditionally manual process but also enhances team productivity, onboarding efficiency, and code comprehension. With future enhancements such as expanded language support, performance optimization, and CI/CD integration, the project sets a strong foundation for modern, intelligent documentation solutions in real-world software development environments.

9.2 Future Work

Future improvements for the project include:

- Expanding multi-language support to specialized languages like Rust, Go, and SQL, and optimizing advanced features such as C++ macros or JavaScript async operations.
- Handling cyclic dependencies, improving performance for large codebases via parallelization and caching, and allowing customization of documentation style and depth are also key areas.
- Integrating with AI-powered code review tools, enhancing search with NLP, offering local and cloud deployment options, enabling real-time documentation updates, and providing AI-assisted code annotations and suggestions can enhance usability.
- Cross-functional integration with CI/CD pipelines and DevOps tools could streamline the development process.

9.3 Applications

1. Enterprise-Level Software Projects: Large organizations with complex, multi-language codebases can use the system to automatically generate and maintain documentation for their software projects. This ensures that documentation stays up to date as the code evolves, reducing the overhead of manual updates.

2. Open Source Projects: The tool can be integrated into open source repositories to automate the generation of documentation. Open source projects often rely on community contributions, and having automated, accurate documentation can make it easier for new contributors to understand the codebase.

3. Continuous Documentation in Agile Development: In fast-paced agile environments, where code changes frequently, this tool can be used to continuously generate updated documentation as part of the development lifecycle. It helps developers stay aligned with changes, ensuring that the documentation always reflects the current state of the code.

4. Legacy System Documentation: In scenarios where legacy systems lack comprehensive documentation, the tool can be applied to generate documentation retroactively. This can be especially valuable in organizations looking to refactor or migrate old codebases where documentation has been lost over time.

5. Code Reviews and Audits: The tool can be used to automatically generate documentation before code reviews or audits. This ensures that auditors or reviewers have a clear understanding of the code's purpose and structure without requiring developers to manually write detailed explanations.

CHAPTER 10

APPENDIX

10.1 Appendix A

10.1.1 Problem Statement Recap

To develop a platform that automates the code documentation process, minimizing manual effort and keeping information consistent and up-to-date. The platform will integrate with the codebase to generate and update documentation automatically, reducing reliance on scattered, outdated tools. It will streamline onboarding, improve productivity, and enhance team collaboration by making code easier to understand and manage.

10.1.2 Satisfiability Analysis

To assess satisfiability, we break the problem down into its core constraints and desired outcomes:

Let:

- D = *Documentation generated automatically*
- C = *Code changes detected accurately*
- A = *Alignment with developer intent*
- M = *Minimal manual intervention*

The platform must satisfy the condition:

$$P = (D \ \hat{} \ C \ \hat{} \ A \ \hat{} \ M)$$

This formula is satisfiable if and only if each subcomponent can be implemented using feasible methods. Based on the use of LLMs (such as GPT-4) and structured parsing algorithms, we observe:

- D is achievable using model inference on well-formed inputs.
- C is solvable using AST diffing, version control hooks, or dependency graphs.
- A is approximate, subject to model accuracy, but can be partially satisfied via prompt engineering.
- M is satisfied by automating repetitive parts and allowing optional overrides.

Therefore, the conjunctive satisfiability condition is logically satisfiable with modern technologies.

10.1.3 Computational Complexity Classification

Component	Problem Type	Class
Code parsing and chunking	Tree traversal / syntax parsing	P
Dependency resolution	Graph traversal / topological sorting	P
Prompt construction for LLMs	Linear string processing	P
Documentation coherence checking	Semantic comparison (heuristic)	NP-Hard
Optimal chunk grouping (to fit LLM context)	Bin packing variant	NP-Hard

Table 10.1: Computational Complexity Classification

10.1.4 Overall Feasibility Verdict

- The problem is logically satisfiable with current AI and compiler technologies.
- Most tasks are solvable in polynomial time (P), with only a few requiring heuristic solutions for NP-Hard subproblems.
- Algebraic modeling confirms that the system's functional pipeline maintains structural integrity across transformations.

10.2 Appendix B

Name of Journal: International Journal of Innovative Research in Technology
Certificate of Publication:

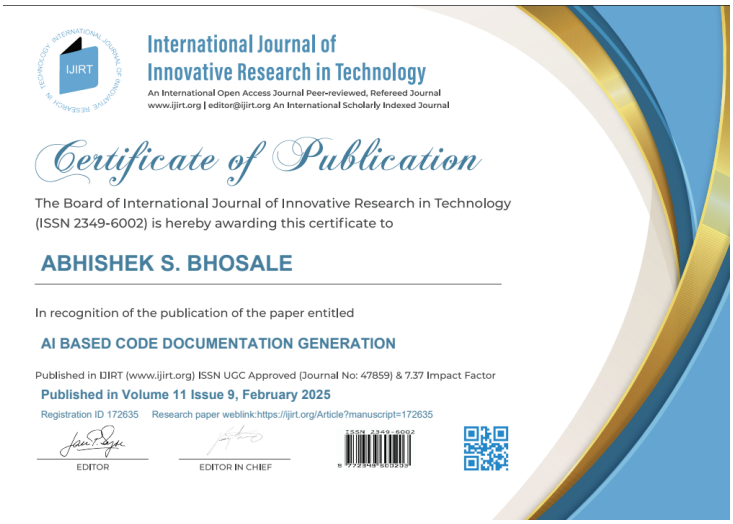
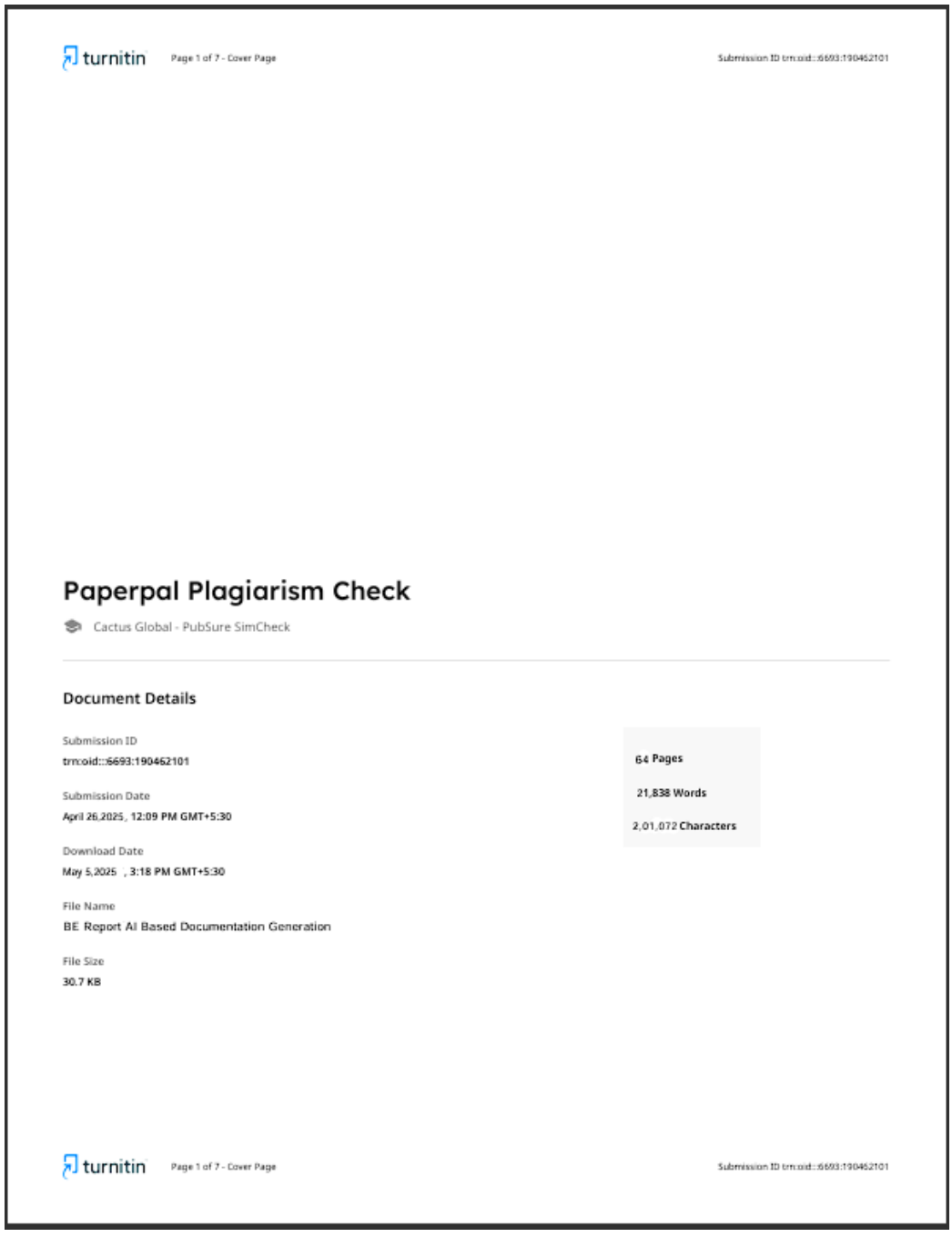


Figure 10.1: Certificate of Publication

10.3 Appendix C


turnitin Page 1 of 7 - Cover Page Submission ID trn:oid:::6693:190462101

Paperpal Plagiarism Check

Cactus Global - PubSure SimCheck

Document Details

Submission ID
trn:oid:::6693:190462101

Submission Date
April 26,2025, 12:09 PM GMT+5:30

Download Date
May 5,2025 , 3:18 PM GMT+5:30

File Name
BE Report AI Based Documentation Generation

File Size
30.7 KB

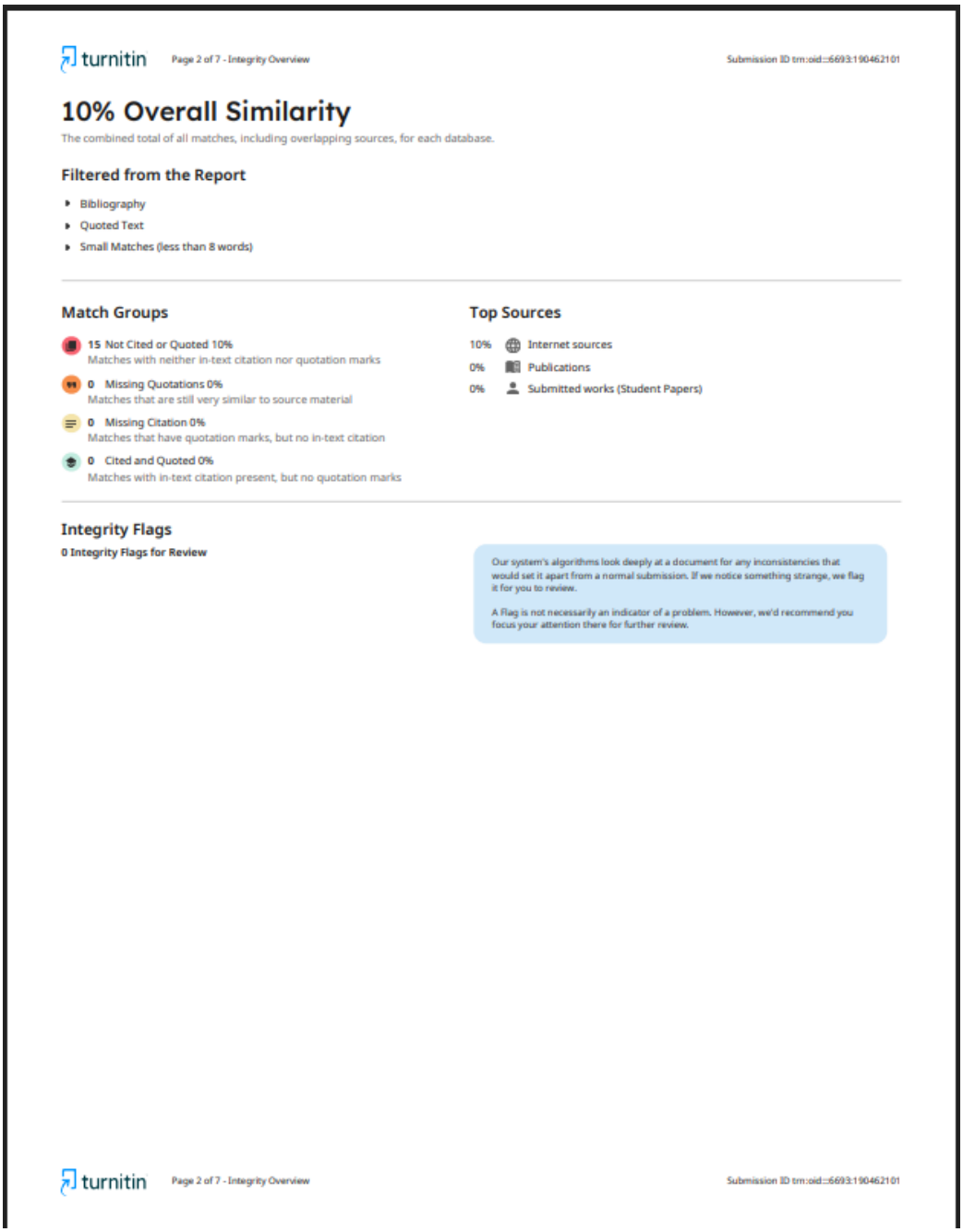
64 Pages

21,838 Words

2,01,072 Characters

turnitin Page 1 of 7 - Cover Page Submission ID trn:oid:::6693:190462101

Figure 10.2: Plagiarism Details 1



turnitin Page 2 of 7 - Integrity Overview Submission ID trn:oid:::6693:190462101

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

15 Not Cited or Quoted 10%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
0% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

turnitin Page 2 of 7 - Integrity Overview Submission ID trn:oid:::6693:190462101

Figure 10.3: Plagiarism Details 2

CHAPTER 11

REFERENCES

[1] McBurney, P. W., & McMillan, C. (2014). Automatic documentation generation. *Proceedings of the International Conference on Software Engineering.*

[2] Nazar, M., Hu, A., & Shihab, E. (2016). Mining software repositories for automated documentation. *Journal of Software Engineering.*

[3] Chen, M., et al. (2021). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374. 5.*

[4] Feng, Z., et al. (2020). CodeBERT: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155.*

[5] Allamanis, M., et al. (2018). A survey of machine learning for big code and naturalness. *ACM Computing Surveys.*

[6] David, S., et al. (2025). Hierarchical Graph-Based Code Summarization for Enhanced Context Retrieval. *arXiv:2504.08975*

[7] Beller, M., Bacchelli, A., & Zaidman, A. (2014). On the impact of test suite metrics on automated documentation. *International Conference on Software Testing, Verification and Validation.*

[8] Zimmermann, T., et al. (2004). Mining version histories to guide software changes. *Proceedings of the International Conference on Software Engineering.*

[9] Rahman, F., Roy, C. K., & Adams, B. (2019). Multi-language software documentation generation. *IEEE Transactions on Software Engineering.*

[10] Wang, Y., et al. (2021). Language-agnostic code analysis with LLMs. *Journal of Software Engineering.*

[11] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *arXiv:1706.03762*

[12] Lu, S., Chowdhury, M. F. M., Huang, P., White, M., & Tian, Y.(2021). CodexGLUE: A benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664.*

[13] Zhang, X., Hou, X., Qiao, X. et al. A review of automatic source code summarization. *Empir Software Eng 29, 162 (2024).*

LIST OF ABBREVATIONS

Abbreviation	Illustration
LLM	Large Language Model
GPT	Generative Pretrained Transformer

LIST OF FIGURES